OCTET 通信协议与数据格式说明 V1.4

产品代号 OCTET（Octal-Channel Transducer & Event Telemetry）；设备上报的 `dev_info.name` 为 `OCTET`。内部型号 DAQ001：存储目录 `/DAQ001/DATA`、录制文件名 与协议 magic `"D205BIN"` 保持不变。

对应固件 1.2.0 · **发布** BI4WMS LABS · 2026-11-12

本文面向需要 **自行开发上位机 / 集成控制 / 数据后处理** 的第三方开发者。 设备默认地址 `192.168.66.55`，TCP 命令/数据端口 `5000`，UDP 发现端口 `5001`， TFTP 文件端口 `69`。

数据存储为 **板载 SD NAND 芯片**（PCBA 集成，非可插拔 SD 卡，FAT32）。 参数掉电存储在 **STM32H753 内部 FLASH**（Bank2 Sector7 @0x081E0000， 日志式追加，64B/条，上电取序号最大的一条）。 所有多字节字段均为**小端**。

1. 帧结构（16 字节头 + payload）

偏移	长度	字段	说明
0	2	magic	固定 `0xA55A`
2	1	ver	协议版本，当前 `0x01`
3	1	type	帧类型，见下表
4	2	seq	序号，用于请求/响应配对
6	2	len	payload 字节数
8	4	param	与帧类型相关（命令码 / 块号 / 事件码）
12	4	crc	`CRC32(帧头 0..11 + payload)`，zlib 兼容（多项式 0xEDB88320）

帧类型

值	类型	方向	param 含义
0x01	CMD	上位机 → 设备	命令码
0x02	RSP	设备 → 上位机	回显命令码
0x03	DATA	设备 → 上位机	块序号 block_index
0x04	EVT	设备 → 上位机	事件码
0x05	DIO	设备 → 上位机	块序号（与 DATA 对齐）

单帧 payload 上限 6500 字节。TCP 上为流式帧，需按头解析并校验 CRC。 CRC 可直接用 Python `binascii.crc32` 计算。

2. 命令（CMD 的 param 字段）

命令	码	payload	说明
PING	0x01	无	连接保活，RSP.value = 运行秒数
GET_INFO	0x02	无	设备信息，见 §4.1
GET_STATUS	0x03	无	运行状态，见 §4.2
GET_NET	0x11	无	读网络参数，见 §4.3
SET_NET	0x12	16 字节	写网络参数，可立即生效
GET_ACQ	0x21	无	读采集参数，见 §4.4
SET_ACQ	0x22	8 字节	写采集参数（采集中禁止）
START	0x23	无	开始采集
STOP	0x24	无	停止采集
SET_TIME	0x31	8 字节	校时，payload = `time_cfg_t`，见 §4.5
GET_TIME	0x32	无	读时间，RSP 后跟 `time_cfg_t`
FILE_LIST	0x41	4 字节	`file_list_req_t`，SD 数据文件列表，见 §4.6
FILE_READ	0x42	56 字节	`file_read_req_t`，分块读文件，见 §4.6
FILE_DEL	0x43	48 字节	`file_del_req_t`，删除文件（采集中禁止）
EVT_DUMP	0x51	0/4 字节	读事件日志（文本行 `time,code,param,msg`）
FACTORY_RESET	0x61	无	恢复出厂设置（IP 回默认）并重启
REBOOT	0x62	无	重启

连接规则

- 同一时刻只允许 **一个** TCP 客户端；第二个连接收到 `busy` 后被关闭
- 长时间空闲建议 5–10s 发一次 PING 保活
- 采集进行中：SET_ACQ / 文件操作返回 `ST_ERR_BUSY (0x05)`

所有响应以 8 字节 `rsp_head` 开头：

```
struct { u8 cmd; u8 status; u16 detail; u32 value; }
```

`status = 0` 表示成功，非 0 见下表：

状态	含义
0x00	成功
0x01	未知命令
0x02	参数非法
0x03	当前状态不允许
0x04	存储器不可用
0x05	忙
0x06	IO 错误
0x07	内存不足

3. 数据帧（DATA）

payload = 20 字节 `data_meta_t` + 采样数据：

```
struct data_meta_t {        // 20 字节
    u32 block_index;        // 块序号，从 0 递增
    u32 start_unix;         // 本块首帧的 Unix 时间
    u32 odr;                // 设备实测采样率 SPS
    u16 frames;             // 本块帧数（默认 256）
    u8  ch_count;           // 使能通道数
    u8  ch_mask;            // 通道位图，bit0 = CH1
    u16 flags;              // bit0 = 本块之前发生过丢块
    u16 reserved;
};
```

采样数据：按帧交错、按使能通道顺序排列，每通道 3 字节（**24bit 有符号小端**）。 一帧占 `ch_count × 3` 字节，8 通道 256 帧一块共 6144 字节。

与存储器 .bin 录制文件的采样格式完全相同（.bin 仅多 52 字节文件头，见 §7）。

4. 关键结构

4.1 dev_info_t（GET_INFO，90 字节）

```
struct {
    char     name[12];       // "OCTET"
    u16 fw_major, fw_minor, fw_patch;
    u16 hw_rev, dev_type;
    u8  mac[6];
    u8  ip[4], mask[4], gw[4];
    u16 tcp_port, udp_port;
    u8  ch_count, ch_mask, odr_index, save_sd, stream_net, din_count;
    u32 duration_ms;
    u32 odr_nominal, odr_measured, serial;   // serial = 芯片 UID 哈希
    char     build[24];      // 编译时间
};
```

4.2 dev_status_t（GET_STATUS，48 字节）

```
struct {
    u8 state;        // 0 启动中 / 1 待机 / 2 采集中 / 3 故障
    u8 fault;        // 故障码，见 §8
    u8 link_up, sd_mounted, recording, streaming, rtc_valid, din_state; // din bit0..3 = DI1..DI4
    u32 uptime_s, frames_total, overrun, rx_drop, free_heap, sd_free_kb, rec_bytes, unix_time;
    u16 rec_files, clients, led_mask, reserved;
};
```

4.3 net_cfg_t（GET_NET / SET_NET，16 字节）

```
struct { u8 ip[4], mask[4], gw[4]; u16 tcp_port; u8 dhcp_en, apply_now; }
```

`apply_now = 1` 时设备保存参数、立即切换 IP 并断开连接， 上位机需用新地址重连。**端口修改后监听会重新绑定。**

4.4 acq_cfg_t（GET_ACQ / SET_ACQ，8 字节）

```
struct { u8 ch_mask, odr_index, save_sd, stream_net; u32 duration_ms; }
```

odr_index	采样率	说明
0	4k SPS	
1	8k SPS	
2	16k SPS	
3	32k SPS	默认
4	64k SPS	
5	128k SPS	
6	256k SPS	

`duration_ms = 0` 表示连续采集；大于 0 时到时自动停止。

4.5 time_cfg_t（SET_TIME / GET_TIME，8 字节）

```
struct { u32 unix_time; int16_t tz_offset_min; u16 reserved; }
```

`unix_time` 为 UTC Unix 秒；`tz_offset_min` 仅用于界面显示，设备内部一律 UTC。

4.6 文件命令结构

```
struct file_list_req_t  { u16 index; u16 count; };            // FILE_LIST
struct file_entry_t     { char name[48]; u32 size; u32 unix_time; };
// FILE_LIST 响应：rsp_head 后紧跟 count 个 file_entry_t（文本行 name,size 亦可）

struct file_read_req_t  { char name[48]; u32 offset; u16 len; u16 reserved; };
struct file_read_rsp_t  { u32 offset; u16 len; u16 eof; u32 total; };
// FILE_READ 响应：rsp_head(8B) + file_read_rsp_t(12B) + 数据(len ≤ 1024)

struct file_del_req_t   { char name[48]; };                   // FILE_DEL
```

`name` 相对于目录 `/DAQ001/DATA`，不能带路径分隔。

5. 事件帧（EVT）与数字输入帧（DIO）

PKT_EVT（0x04）payload，40 字节：采集启停、故障、配置变更等异步事件。

```
struct evt_payload_t {
    u16 code;            // 事件码（与设备事件日志一致）
    u16 arg;
    u32 unix_time;
    char text[32];       // 文本描述
};
```

PKT_DIO（0x05）payload，28 字节：每产生一个数据块，随 PKT_DATA 之后发送一帧， 上报 4 路数字输入快照。上位机只用于显示，不写 WAV/SD。

```
struct dio_meta_t {
    u32 block_index;     // 与同块 PKT_DATA 的 block_index 对齐
    u32 unix_time;
    u8  state;           // bit0..3 = DI1..DI4 当前有效电平
    u8  din_count;       // 固定 4
    u16 reserved;
    u32 edges[4];        // 各通道累计上升沿计数（自上电）
};
```

设备侧 DI 在 1ms 任务里采样（5ms 采样节拍 + 3 次消抖）。

6. 典型交互

```
上位机                                  设备
   |---- CMD(SET_ACQ, ch=0xFF, 32k) ---->|
   |<--- RSP(SET_ACQ, ok) --------------|
   |---- CMD(START) -------------------->|
   |<--- RSP(START, ok) ----------------|
   |<=== DATA(block0) ================== |
   |<=== DIO(block0) ==================== |   数字输入快照（每块一帧）
   |<=== DATA(block1) ================== |   ... 采集期间连续推流
   |---- CMD(STOP) --------------------->|
   |<--- RSP(STOP, ok) -----------------|
```

最小 Python 客户端示例

```
import socket, struct, binascii

def crc(b): return binascii.crc32(b) & 0xFFFFFFFF

def frame(t, seq, param, payload=b""):
    hdr = struct.pack("<HBBHHII", 0xA55A, 1, t, seq, len(payload), param, 0)
    return hdr + payload + struct.pack("<I", crc(hdr + payload))

def parse(buf):
    magic, ver, t, seq, ln, param = struct.unpack("<HBBHHI", buf[:12])
    body = buf[12:12+ln]
    (crc_,) = struct.unpack("<I", buf[12+ln:16+ln])
    assert crc(buf[:12] + body) == crc_
    return t, param, body

s = socket.create_connection(("192.168.66.55", 5000))
s.sendall(frame(0x01, 1, 0x02))             # CMD GET_INFO
t, param, body = parse(recv_frame(s))       # body[0:8] = rsp_head, 其后 dev_info_t
```

7. 存储器数据文件（板载 SD NAND）

目录 `/DAQ001/DATA/`，文件名 `<时间戳>_<N>CH_<ODR>sps.bin`。

- 文件头 52 字节 `rec_header_t`（`magic = "D205BIN"`，Python 解包格式 `"<8sHBBIIIHH6I"`，含通道数、掩码、标称 ODR、起始 Unix 时间、总帧数、 每样本 3 字节）
- 紧随其后为帧交错 24bit 小端采样，与 DATA 帧的采样格式一致
- 停止录制时生成同名 `.json` 元数据（device / firmware / channels / channel_mask / odr / bits=24 / format="int24le-interleaved" / frames / duration_s / start_unix / end_unix）

转 WAV 只需按 `nchannels=ch_count, sampwidth=3, framerate=odr` 逐帧写入。

8. 故障码

码	名称	含义
0	NONE	无故障
1	SD_MISSING	存储器未就绪
2	SD_IO	SD 读写错误
3	ADC_NO_DATA	AD7768 无数据输出
4	ADC_CFG	ADC 配置回读校验失败
5	（保留）	历史参数存储损坏
6	ETH_LINK	网线未连接
7	OVERRUN	采集数据溢出
8	OVERTEMP	温度过高
9	（保留）	历史外部 NOR 错误
10	RTC	RTC 掉电 / 通信异常

9. TFTP 文件通道（UDP 69）

设备内置 RFC1350 服务端，用于批量拷贝板载存储上的录制文件；支持 `blksize` 选项协商（上限 1428 字节，不产生 IP 分片）。传输目录固定为 `/DAQ001/DATA`，文件名里不能带路径。

操作	报文	说明
下载（设备 → PC）	RRQ	读文件；文件不存在回 ERROR 1
上传（PC → 设备）	WRQ	写文件；同名已存在回 ERROR 6
采集/录制期间	任意请求	回 ERROR 2 `device busy (acquiring)`

要点：

- 停止等待协议，块超时 3s、重传 6 次；单客户端串行处理
- 应答从 69 端口发出（多数客户端以首包源端口为 TID，可直接使用）
- TFTP 协议本身没有删除语义，删除仍走 TCP `CMD_FILE_DEL (0x43)`

Windows 自带客户端示例：

```
tftp -i 192.168.66.55 GET 20260101_101010_8CH_32000sps.bin d://tmp//a.bin
tftp -i 192.168.66.55 PUT d://tmp//cal.bin cal.bin
```

10. UDP 发现协议（端口 5001）

上位机向设备 UDP 5001 端口发送任意数据包（推荐发一个 `CMD(PING)` 帧便于校验）， 设备立即从收到包的源地址回一帧协议应答：8 字节 `rsp_head_t` （cmd=GET_INFO, status=OK）+ 90 字节 `dev_info_t`。

整个 UDP 报文 同样按 §1 帧结构封装（16B 头 + payload + CRC）。据此可扫描网段自动发现设备。

11. 参考实现

完整可运行的参考实现（协议栈、CRC、TFTP 客户端、bin→WAV 转换）见 `host/daq001_tool.py`；GUI 上位机见 `host/daq001_gui.py`， 可直接作为第三方二次开发的起点。